

SZEGEDI TUDOMÁNYEGYETEM

Természettudományi és Informatikai Kar

Számítástudomány Alapjai Tanszék

Szakdolgozat

Genetikus algoritmus fejlesztése a heterogén járműútvonal-tervezési problémára

Developing a Genetic Algorithm for the Heterogeneous Vehicle Routing
Problem

Vizi Fanni

Programtervező informatikus BSc

Témavezető: **Békési József Dr.**

főiskolai tanár

Szeged

2026

Feladatkiírás

A szakdolgozat célja a járműútvonal-tervezési problémák (Vehicle Routing Problem, VRP) egy speciális változatának vizsgálata, amelyben a rendelkezésre álló járműflotta heterogén, azaz különböző kapacitással rendelkező járművekből áll.

A dolgozat keretében a hallgató feladata:

A klasszikus VRP és a heterogén flottás VRP (Heterogeneous Vehicle Routing Problem, HVRP) szakirodalmának áttekintése

A savings algoritmus (vagyis a Clarke–Wright-féle módszer) működésének ismertetése és alkalmazási lehetőségeinek vizsgálata heterogén járművek esetén

Genetikus algoritmus alapú megoldási módszerek fejlesztése a probléma kezelésére

A két megközelítés (savings alapú heurisztika és genetikus algoritmus) implementálása a heterogén járműflottára

Kísérleti vizsgálatok elvégzése különböző benchmark adatokon

Az algoritmusok teljesítményének összehasonlító elemzése (megoldás minősége, futási idő, stb.)

Következtetések levonása és további fejlesztési irányok megfogalmazása

Elvárt eredmények:

Működő algoritmusok implementációja

Összehasonlító elemzés és kiértékelés

A módszerek előnyeinek és korlátainak feltárása

Dokumentált számítási eredmények és következtetések

Tartalmi összefoglaló

A szakdolgozatom célja heterogén járműútvonal-tervezési probléma megoldása genetikus algoritmussal.

A járműútvonal-tervezési probléma (Vehicle Routing Problem, VRP) egy olyan NP-nehez optimalizálási probléma, ahol egy minimális költségű útvonalhalmazt keresünk egy adott járműflottával, úgy, hogy minden vevőt kiszolgáljunk.

A VRP egyik alproblémája a heterogén járműútvonal-tervezési probléma (Heterogeneous Vehicle Routing Problem, HVRP), ahol a flotta különböző kapacitású járművekből áll. A probléma megoldásához a savings módszert, valamint a genetikus algoritmust használtam.

A Clarke-Wright féle savings algoritmus egy népszerű heurisztika a VRP problémák megoldására. Az alapvető gondolat az algoritmus mögött az, hogy a teljes költség minimalizálását az útvonalak összevonásával próbáljuk meg elérni, és mindig azt a két utat vonjuk össze, amivel a legnagyobb megtakarítást érjük el.

A genetikus algoritmus egy populáció alapú metaheurisztika, ami sokféle problémára alkalmazható. Az algoritmus alapja az evolúció és a természetes szelekció. Egy iterációban az aktuális generációból egy újat állítunk elő a szelekciós operátor által kiválasztott egyedek keresztezésével, és az utódok mutálásával. Mivel mindig a legrátermettebb egyedeket választjuk szülőknek, elméletben minden generációval egyre jobb megoldásokat kapunk.

Az implementációt a JetBrains IntelliJ IDEA fejlesztői környezetében, Java nyelven írtam meg, verziókövetésre a GitHub-ot használtam.

Összességében elmondható, hogy a tesztelési adatokon a savings módszer érte el a legjobb megoldásokat. Ezen megoldások minőségét lokális kereséssel vagy a genetikus algoritmus alkalmazásával még kis mértékben lehet javítani.

Kulcsszavak: vrp, hvrp, savings, genetikus, java

Tartalomjegyzék

Feladatkiírás.....	2
Tartalmi összefoglaló.....	3
Tartalomjegyzék	4
1. Bevezetés.....	6
1.1 A járműútvonal-tervezési probléma	6
1.2 A heterogén járműútvonal-tervezési probléma	6
1.3 Megoldási módszerek a VRP-re.....	7
2. Felhasznált eszközök és módszerek	8
2.1 A savings algoritmus	8
2.2 A genetikus algoritmus.....	9
2.2.1 Az algoritmus alapjai	9
2.2.2 A szelekciós operátor	9
2.2.3 A keresztező operátor	10
2.2.4 A mutációs operátor	11
2.3 Lokális keresés	11
2.4 PyVRP adatok	12
2.5 Fejlesztési környezet	12
3. Megvalósítás.....	13
3.1 A probléma modellezése	13
3.2 A savings algoritmus módosítása heterogén járművekre	14
3.2.1 A CWSavings és a Saving osztály.....	14
3.2.2 Az algoritmus megvalósítása	14
3.2.3 A járművek kiválasztása.....	15
3.2.4 Az utak összevonása.....	15
3.3 A genetikus algoritmus.....	16
3.3.1 Kromoszómák kódolása és dekódolása.....	16

3.3.2	Kezdeti populáció kialakítása.....	17
3.3.3	Kiválasztó operátorok	17
3.3.4	Keresztező operátorok.....	18
3.3.5	Mutációs operátorok.....	20
3.3.6	Az utak helyreállítása és értékelése.....	20
3.3.7	Az algoritmus paraméterei	21
3.4	Megoldások javításai.....	22
3.4.1	Savings javítása lokális kereséssel	22
3.4.2	Savings javítása genetikus algoritmussal	23
4.	Eredmények.....	24
4.1	Tesztkörnyezet.....	24
4.2	Savings algoritmus teljesítménye.....	24
4.3	A genetikus algoritmus.....	25
4.4	Savings javításai.....	26
4.5	Konklúzió.....	27
5.	További lehetőségek.....	28
	Irodalomjegyzék	29
	Nyilatkozat.....	30

1. Bevezetés

1.1 A járműútvonal-tervezési probléma

A járműútvonal-tervezési probléma (Vehicle Routing Problem, VRP) egy kombinatorikai optimalizálási probléma. A probléma célja egy minden vevőt kiszolgáló optimális útvonal keresése a megadott járműflottával. [1]

A problémát modellezhetjük egy gráfként, ahol a csúcsok a vevők, illetve a kiindulási raktár, az élek súlya pedig a csúcsok közötti költség. Minden vevőnek van egy megadott igénye, vagy más néven kereslete, amit a kiszolgáló járműnek ki kell tudnia elégíteni. A költséget számíthatjuk a csúcsok távolságaként, de lehet más költségfüggvényünk is, amely figyelembe veszi a járművek költségeit is.

Az alapprobléma megoldásakor több korlátozást figyelembe kell vennünk. Minden vevőt pontosan egyszer kell kiszolgáltatnunk, illetve minden útvonal a raktárból indul és oda is tér vissza. Feltesszük továbbá, hogy ismerjük a csúcsok keresletét, a csúcsok távolságát és a járművek költségeit. A cél a teljes költség minimalizálása a korlátozások betartása mellett. [2]

A VRP egy NP-nehéz probléma, ezért az egzakt algoritmusok csak korlátozottan használhatók rá a nagy műveletigény miatt. Ebből kifolyólag nagyobb gráfokra heurisztikus vagy metaheurisztikus módszereket szokás használni.

A VRP-nek nagyon sok alproblémája van, amelyek további korlátozásokat vezetnek be az alap problémára. Ilyen a kapacitált VRP (Capacitated Vehicle Routing Problem, CVRP), ahol minden járműnek van egy maximális kapacitása, vagy az időablakos VRP, ahol vannak vevők, amiket csak megadott időablakon belül tudunk meglátogatni.

A CVRP a következő korlátozásokkal egészíti ki az alap problémát:

- A raktár kereslete nulla
- A vevők kereslete nem osztható fel
- A járművek nem léphetik túl a maximum kapacitásukat

1.2 A heterogén járműútvonal-tervezési probléma

A heterogén járműútvonal-tervezési probléma (Heterogeneous Vehicle Routing Problem, HVRP) egy olyan kapacitált járműútvonal-tervezési probléma, ahol a járműflotta járműveinek különböző maximális kapacitása van.

A fő különbség az alapproblémához képest, hogy a járműveknek nem csak más kapacitása, hanem más költségei is vannak. A HVRP-ben minden járműnél megkülönböztetünk egy fix és egy változó költséget. A fix költség független attól, hogy mekkora utat tesz meg a jármű, minden esetben ugyanannyi, ha a járművet használjuk. A változó költség pedig az egységnyi út költségét adja meg az adott járműre.

Ennek a problémának is vannak különböző variánsai, például az alapján, hogy a járművek száma korlátozott-e. A szakdolgozatban egy olyan alproblémát oldok meg, amelyben minden járműtípusból megadott számú áll rendelkezésre.

A HVRP alapvetően egy összetettebb probléma, mint az alap VRP, mivel az optimális út meghatározásán túl szükség van a megfelelő jármű kiválasztására az úthoz.

1.3 Megoldási módszerek a VRP-re

A VRP-t megoldó algoritmusok három fő csoportra oszthatók: az egzakt módszerek, a heurisztikák és a metaheurisztikák.

Az egzakt módszerek mindig optimális megoldást adnak, viszont a nagy műveletigényük miatt nem skálázódnak jól, így a gyakorlatban ritkán használhatók.

A heurisztikus módszerek célja ezzel szemben egy jó, de nem optimális megoldás gyors keresése. Éppen ezért jobban alkalmazhatóak a gyakorlatban, az algoritmusok viszont sokszor probléma-specifikusak, illetve beragadhatnak lokális optimumba. Ebbe a kategóriába tartozik a később tárgyalt savings algoritmus is.

A metaheurisztikák általános problémamegoldó keretrendszerek, amelyek a heurisztikákhoz hasonlóan egy jó, nem optimális megoldás gyors keresésére alkalmasak. Nagy problémákra jobb megoldásokat adhatnak, mivel kevésbé hajlamosak a beragadásra. Ide tartoznak például a genetikus algoritmusok.

2. Felhasznált eszközök és módszerek

2.1 A savings algoritmus

A savings algoritmus egy konstruktív heurisztika a kapacitált VRP (Capacitated Vehicle Routing Problem, CVRP) megoldására, amelyet Clarke és Wright vezetett be 1964-ben [3]. Az algoritmus alapötlete, hogy úgy vonunk össze utakat, hogy a lehető legnagyobb megtakarítást kapjuk.

Az algoritmus egy alap megoldásból indul ki, ahol minden vevőhöz egy saját utat rendelünk, ahol a jármű a raktárból a vevőhöz megy, majd vissza. Az algoritmus lépései a következők: számoljuk ki a megtakarítást minden vevőpárra, rakjuk a megtakarításokat csökkenő sorrendbe, majd menjünk végig a megtakarításokon és vonjuk össze az utakat, ha teljesítik a megadott feltételeket.

A megtakarítás számítása a következő képlet alapján történik:

$$s(i, j) = d(D, i) + d(D, j) - d(i, j) \quad (1)$$

Az (1) képletben i és j a vevők, D a raktár, $s(i, j)$ az (i, j) vevőpár megtakarítása, $d()$ pedig a távolságfüggvény, ami megadja a két pont közötti távolságot.

Két utat akkor vonhatunk össze, ha teljesítik a következő három feltételt:

- Nem sértik a CVRP korlátozásait, tehát az úton érintett vevők összigenye nem haladja meg a jármű maximum kapacitását.
- A két vevő két különböző úton van.
- Egyik vevő sem belső csúcs a saját útján, vagyis közvetlenül kapcsolódnak a raktárhoz.

Az algoritmus akkor áll meg, ha végig értünk az összes (i, j) vevőpár megtakarításán.

A savings algoritmus az egyik legelterjedtebb heurisztika a VRP és különböző variánsainak megoldására, mivel könnyen implementálható, gyors és nagyobb problémákra is alkalmazható. Hátránya, hogy be tud ragadni lokális optimumokba.

2.2 A genetikus algoritmus

2.2.1 Az algoritmus alapjai

A genetikus algoritmusok olyan populáció alapú metaheurisztikák, amelyek a természetes szelekción alapulnak. Az algoritmus egy kezdeti populációból indul ki, majd a jobb megoldások keresztezésével minden generációban javítja az eredményt.

A kezdeti populáció egyedei lehetnek random megoldások, vagy valamilyen heurisztikával előállítottak is. Minden egyed egy megoldást reprezentál, ez a reprezentáció lesz az egyed kromoszómája.

Minden egyedhez hozzárendelünk egy fitness értéket is. Ez az érték nagyban függ a megoldandó problémától. A VRP esetében a fitness általában a költség, esetlegesen valamilyen büntetéssel érvénytelen megoldások esetén. [4]

A genetikus algoritmusoknak három fő operátora van: a szelekció, a keresztezés és a mutáció. A szelekció kiválasztja a szülőket a következő generációhoz, általában a rátermettség alapján. A keresztezés létrehozza az utódokat a szülők kromoszómáinak valamilyen kombinációjával. A mutáció az utódok génjeit kis mértékben módosítja, a célja, hogy megmaradjon a diverzitás a populációban.

A genetikus algoritmusok megállási feltétele lehet egy megadott generációs szám, vagy megszabhatunk valamilyen feltételt a generációk közötti legjobb vagy átlagos fitnessértékek különbsége alapján.

A genetikus algoritmusok előnye, hogy általános problémamegoldók, nagyon sok különböző területen használhatók, illetve nagyon jók összetett, nagy problémák megoldásában.

2.2.2 A szelekciós operátor

A szelekciós operátorok választják ki a szülőket a következő generációhoz. A szelekció fontos a genetikus algoritmusokban, mivel a legrátermettebb egyedek keresztezésével tudjuk elérni, hogy a megoldásaink generációról generációra javuljanak. A következőkben három szelekciós operátort fogok bemutatni.

Az első-n szelekció (First-N Selection) az egyik legegyszerűbb szelekciós stratégia. Lényegében az n legjobb egyedet választjuk ki a populációból, és őket keresztezzük. Ennek az operátornak a hátránya, hogy kevésbé tartja meg a populáció diverzitását, mint más módszerek, ezért korai konvergációt okozhat, viszont nagyon könnyen implementálható.

A rulett szelekció (Roulette Selection) lényegében egy rulettkerék működését szimulálja. A keréken minden egyedhez a fitnessértékével arányos szeletet rendelünk, majd egy random szám generálásával szimuláljuk a kerék pörgetését, és kiválasztjuk a megfelelő egyedet. Az algoritmus olyan esetekben használatos, ahol a fitnessértéket maximalizálni szeretnénk, illetve az egyedek fitnessértékei nagy mértékben eltérnek egymástól.

A versenyalapú szelekciónál (Tournament Selection) először kiválasztunk n darab egyedet, majd ezek közül a legjobb nyer. Ez a legjobb érték lehet a legkisebb, ha minimalizálni szeretnénk, vagy a legnagyobb, ha maximalizálás a cél. Ez a módszer alkalmazható tehát minimalizálásra és maximalizálásra is, valamint jó választás akkor is, ha az egyedek fitnessértékei közel vannak egymáshoz.

2.2.3 A keresztező operátor

A keresztező operátorok feladata a két szülő génjeinek kombinálása, hogy új egyedeket kapjunk. A keresztezés célja, hogy az egyedek jó tulajdonságait megtartsuk és kombináljuk, ezzel gyorsítva az algoritmus konvergenciáját. A keresztező operátoroknak több csoportja létezik a kromoszómák sajátosságai alapján. A VRP megoldások reprezentációja permutáció alapú, ezért ennek megfelelő operátorokat kell használnunk.

A ciklikus keresztezés (Cycle Crossover, CX) egy egyszerű keresztező operátor permutációs feladatokra. Elsődleges célja, hogy megőrizze a gének abszolút pozícióját, ez az operátor okozza a legkisebb változást a kromoszómákban. Az algoritmus lényege, hogy ciklusokat keres a két szülő között, hogy elkerüljük a kihagyott vagy duplikált elemeket. Először választunk egy pontot az egyik szülőben, majd megkeressük a hozzá tartozó ciklust a két szülő között. A ciklusban lévő értékeket az első szülőből vesszük át, a cikluson kívülieket a másiktól.

A sorrend alapú keresztezés (Order Crossover, OX) célja, hogy megőrizze a gének relatív sorrendjét. Az operátor jól használható olyan problémákra, mint az útvonaltervezés, ahol jobban számít a sorrend, mint az abszolút pozíció. Ebben a megközelítésben két keresztezési pontot jelölünk ki. A két pont közötti elemeket egyszerűen átmásoljuk az egyik szülőből, majd a még kimaradt elemeket olyan sorrendben illesztjük be, ahogyan a másik szülőben szerepelnek.

A részleges leképezésű keresztezés (Partially Mapped Crossover, PMX) a két szülő közötti leképezésekre hagyatkozik a duplikáció elkerülésére. Ez a módszer, ahol lehetséges megőrzi a gének abszolút pozícióját, a konfliktusokat pedig szabályok alapján oldja fel, ezzel nagyobb különbségeket okoz, mint az OX. A PMX implementálása bonyolultabb, viszont ütemezési

feladatoknál ez lehet a legjobb választás. Első körben itt is átmásoljuk a két keresztezési pont közötti szakaszt az egyik szülőből. Ezután elkezdjük a kimaradt pozíciókon lévő értékeket átmásolni. Amennyiben konfliktus lép fel, a leképezések segítségével fel tudjuk őket oldani.

2.2.4 A mutációs operátor

A mutációs operátorok célja, hogy megmaradjon a populáció diverzitása, és ne ragadjon be az algoritmus egy lokális optimumba. Nem minden utódra alkalmazunk mutációt, általában csak kis eséllyel következik be, mivel a túl nagy mutációs ráta elronthatja a jó megoldásokat, ezzel lassítja a konvergenciát. A permutációs feladatokra a keresztező operátorokhoz hasonlóan más módszereket kell alkalmaznunk, mint általános esetben.

A csere mutáció (Swap Mutation) a legegyszerűbb ilyen operátor, csak két pont értékét cseréli meg a kromoszómában. A módszer minimális változást okoz, ezért elsődleges szerepe a jó megoldások finomítása.

A keverő mutáció (Scramble Mutation) a kromoszóma egy szakaszát véletlenszerűen keveri össze. Ez nagyobb változással jár, jobban felfedezi a keresési teret, viszont a jó részsorozatok teljesen el tudja rontani.

Az inverziós mutáció (Inversion Mutation) szintén a kromoszóma egy szakaszával dolgozik, aminek megfordítja a sorrendjét. Az operátor megőrzi a szakaszon belül a relatív sorrendet, ezért egyszerű útvonal-tervezésnél hatékony, viszont olyan feladatoknál, ahol fontos lehet az abszolút pozíció, ronthatja a megoldásokat.

2.3 Lokális keresés

A keresés egy olyan folyamat, ahol egy állapottérben valamilyen stratégia alapján célállapotokat keresünk. Az állapottér egy olyan gráf alapú reprezentációja egy adott feladatnak, ahol a gráf csúcsai a feladat lehetséges állapotai, az élek pedig az átmenetek az állapotok között. Egy állapot szomszédai tehát pontosan azok az állapotok, amelyeket egy lépésből el tudunk érni.

A lokális keresés egy olyan mohó algoritmus, ahol a keresés mindig a szomszédai közül választja ki a következő lépést. A lokális keresést használhatjuk meglévő megoldások javítására.

2.4 PyVRP adatok

A PyVRP egy Python alapú, nyílt forráskódú eszköz a különböző VRP variánsok megoldására. A könyvtár tartalmazza a PyVRP megoldó csomagot, a VRPLIB olvasó és író csomagot, egy Bench benchmarking eszközt, illetve a példákat az Instances könyvtárban. A szakdolgozatomhoz az Instances könyvtár példáit használtam.

A könyvtárban 10 különböző VRP alproblémára találhatók megoldások, köztük a HVRP-re is. A HVRP adatok a Pessoa et al. [5] által publikált cikk kiegészítő anyagából származnak. Ezek a példák az Uchoa et al. és társai [6] által javasolt CVRP példák alapján készültek, a generálás menetét a 2018-as cikk részletezi.

Az adatok választásakor fontosnak tartottam, hogy olyan adathalmazon dolgozzak, amiben különböző méretű példák vannak, hogy lássam, hogyan skálázódik a megoldásom. Ebben a könyvtárban külön fájlban vannak a példák, azonos formátumban, többek között az ebből adódó könnyű beolvasás miatt választottam.

Az adatok tartalmazzák a vevők, mint csúcsok koordinátáit és igényüket, valamint a járművek kapacitását, fix- és változó költségeit. A raktár csúcs is meg van adva.

2.5 Fejlesztési környezet

Az implementációhoz a Java nyelvet választottam főleg az objektumorientáltsága miatt, valamint ebben a nyelvben vagyok a legjártasabb.

A fejlesztéshez az IntelliJ IDEA fejlesztői környezetet használtam, a verziókövetést GitHub-bal végeztem, a könyvtár elérhető a <https://github.com/fannivizi/hfvrp> linken.

3. Megvalósítás

3.1 A probléma modellezése

A probléma modellezéséhez szükségünk van a vevők, a járművek, valamint az utak reprezentációjára is. A vevők kezelését a Node osztály, a járművekét a Vehicle osztály, az utakét pedig a Route osztály végzi.

A Node osztály minden vevőről három adatot tárol, a vevők indexét és az igényét egész számként, illetve a koordinátáit egy Coordinate objektumban. A Coordinate osztály egy x és y koordinátát tárol egész számként. Ez az osztály valósítja meg a távolságfüggvényt, amely megadja két Node euklideszi távolságát, melynek képlete:

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (2)$$

ahol $d(i, j)$ az (i, j) vevőpár távolsága, i koordinátái (x_i, y_i) , j koordinátái pedig (x_j, y_j)

A Vehicle osztály tárolja a járművek indexét és kapacitását egész számként, valamint a jármű fix- és változó költségét egy-egy lebegőpontos változóban.

A Route osztály az úthoz tartozó Vehicle objektumot (*vehicle*), valamint egy Node listát (*nodes*) tartalmaz. Az osztály három legfontosabb függvénye a distance(), a demand() és a totalCost(), amelyek visszaadják rendre az út teljes hosszát, az úton érintett vevők összesített keresletét, valamint az út összköltségét. Az összköltség számítása az

$$F + U * d \quad (3)$$

képlet alapján történik, ahol F a jármű fix költsége, U a jármű változó költsége, a d pedig a distance() függvény által számított távolság. Technikai szempontból fontos megemlíteni, hogy minden út vevőlistájában az első és utolsó elem a raktár, a továbbiakban viszont a lista elején és végén mindig az első, illetve az utolsó vevőt értjük, nem a raktárat.

A megoldások kiértékelését a Statistics osztály végzi, amely egy adott Route listára kiszámolja az érintett vevők számát, a használt járművek számát, az utak összesített távolságát, valamint az utak összköltségét. A későbbiekben ez segít az algoritmusok összehasonlításában.

3.2 A savings algoritmus módosítása heterogén járművekre

3.2.1 A CWSavings és a Saving osztály

A savings algoritmust a CWSavings osztály valósítja meg. Az osztály tárolja a flotta, a vevők, illetve a raktár másolatát, valamint a megoldást a *routes* listában.

Az osztályban két segéd függvény található, az *interior()*, ami megmondja, hogy az adott csúcs egy adott úton belső csúcs-e, illetve a *merge()*, ami két külön út összevonását végzi.

A megtakarítások megfelelő tárolásához létrehoztam egy Saving osztályt is, amely együtt tárolja a megtakarítást (*saving*), illetve a csúcspárt (a, b), amihez tartozik az érték.

Az algoritmus a klasszikus savings algoritmus lépéseit követi, néhány módosítással, hogy alkalmas legyen a HVRP megoldására.

3.2.2 Az algoritmus megvalósítása

Először létre kell hoznunk a kezdeti megoldást, ahol minden vevő külön úton van. Ehhez végigiterálunk a vevőkön, majd minden N csúcsához létrehozunk egy új Route változót, ahol a *vehicle* értéke null, a *nodes* lista pedig a raktár – N – raktár csúcsokat tartalmazza ebben a sorrendben.

A következő lépés a megtakarítások kiszámítása. Ehhez két for ciklust használunk, legyen n a vevők száma, i a külső ciklus ciklusváltozója, j pedig a belső ciklusé. Ekkor a külső ciklus 0-tól n -ig fut, a belső pedig $i + 1$ -től n -ig. Ezzel végigmegyünk az összes lehetséges, páronként különböző vevőkombináción. Minden (n_i, n_j) vevőpárosra hozzunk létre egy Saving változót, ahol az a értéke n_i , b értéke n_j , a *saving* értéke pedig a korábban említett képlet alapján számolt érték. Ezeket a változókat tegyük egy listába, majd rendezzük őket csökkenő sorrendbe a *saving* érték alapján.

Ezután iteráljunk végig az összes Saving változón. Használjuk a két csúcsra az a és b , a megtakarítás értékére az s jelölést. Ellenőrizzük, hogy teljesülnek-e a feltételek az utak összevonására, ehhez először keressük meg, hogy a és b melyik utakon vannak. Ha ez a két út ugyanaz, vagy bármelyik vevő belső csúcs a saját útján, akkor már biztosan megsértünk legalább egy követelményt, ezért folytassuk a következő Saving változóval.

3.2.3 A járművek kiválasztása

A HVRP probléma sajátossága az, hogy minden úthoz ki kell választanunk a megfelelő járművet. Legyen r_a és r_b a két út, amin a és b található. Három esetet kell vizsgálnunk: egyik úthoz sem rendeltünk még járművet, pontosan egy úthoz rendeltünk járművet vagy már mindkét úthoz rendeltünk járművet.

Ha még egyik úthoz sem rendeltünk járművet, akkor keressük meg a legkisebb olyan kapacitású járművet, ahol a két út összkereslete nem haladja meg ezt a kapacitást. Amennyiben van ilyen v jármű, távolítsuk el v -t a flottából, majd végezzük el az összevonást r_a -ra és r_b -re a v járművel.

Ha az egyik úthoz már tartozik jármű, akkor állapítsuk meg, melyik ez az út. Ha az r_a úthoz tartozó jármű v_a , r_b -hez pedig nem tartozik jármű, akkor ellenőrizzük, hogy r_a és r_b összkereslete meghaladja-e v_a kapacitását. Ha igen, akkor nem tudjuk összevonni a két utat, mivel sértenénk a HVRP korlátozásait, ezért folytassuk a következő Saving változóval. Ha viszont nem haladja meg az összkereslet a kapacitást, akkor végezzük el az összevonást r_a -ra és r_b -re a v_a járművel. Fordított esetben, ahol r_b -hez a v_b jármű tartozik, r_a -hoz pedig null, ugyanezeket a lépéseket hajtjuk végre, csak v_a helyére a v_b -t helyettesítjük.

Amennyiben mindkét úthoz rendeltünk járművet, ezeket ne vonjuk össze, lépünk a következő Saving értékre.

Ha végigértünk az összes megtakarításon, és vannak még járművek, amiket nem használtunk fel, akkor próbáljuk meg őket párosítani olyan utakkal, amelyekhez nem rendeltünk járművet. Ezzel javíthatjuk a megoldást, de ez sem garantálja, hogy minden úthoz tudunk járművet rendelni.

3.2.4 Az utak összevonása

Az utak összevonását a `merge()` metódus végzi. A metódus argumentumként megkapja a megtakarításhoz tartozó a és b csúcsokat, a két összevonandó utat, amin a és b található: r_a és r_b , valamint a v járművet.

Először el kell távolítanunk az r_a és r_b utakat a megoldásból. Ezután ellenőrizzük, hogy a r_a és r_b utakon a és b milyen pozícióban szerepel. Mivel már korábban tudjuk, hogy sem a , sem b nem belső csúcs, ehhez elég megnéznünk, hogy az első pozíción szerepelnek-e. Amennyiben az a nem az utolsó helyen áll, akkor megfordítjuk r_a -ban a vevők sorrendjét, illetve, ha b nem az első helyen áll, akkor megfordítjuk r_b sorrendjét. Erre azért van szükség,

mert a megtakarítás értéke arra az esetre vonatkozik, amikor a -ból b -be közvetlen kapcsolat van, ezért a -nak r_a végén, b -nek r_b elején kell lennie.

Ha elvégeztük a megfordításokat, akkor létrehozhatjuk az új Route változót a v járművel, majd a nodes listába belerakjuk sorrendben először az r_a , majd az r_b elemeit. Az új utat végül belerakjuk a megoldásba.

3.3 A genetikus algoritmus

A genetikus algoritmus futtatását a Genetic osztály végzi. Az osztály tárolja a flotta és a vevők másolatát, a raktárat, illetve a populáció nagyságát, magát a populációt egy listában, a generációk számát, valamint egy random változót.

Az egyedek tárolásához létrehoztam az Individual osztályt, amely tárolja az egyed kromoszómáját, vagyis a megoldás kódolt változatát, illetve a hozzá tartozó fitnessértéket.

3.3.1 Kromoszómák kódolása és dekódolása

A megoldások kódolását a code() metódus végzi. A függvény bemenete az utak listája, kimenete pedig egy egyed. Az algoritmus végigmegy a megoldás útjainak listáján, és a vevők indexét sorrendben beleteszi egy listába. Amikor az út végére ér, akkor a raktár indexe helyett az úthoz tartozó jármű indexének negáltját teszi a listába, ezzel jelezve, hogy itt ér véget az út. Itt azért a negáltat használjuk, mivel a járművek és a vevők indexelése is 1-től kezdődik, így tudjuk megkülönböztetni a vevőket az utakat elkülönítő markerektől. Mivel minden kódolt megoldásnak ugyanolyan hosszúnak kell lennie, a nem használt járművek negált indexét is beillesztjük a lista végére. Ha megvan a kódolt kromoszóma, akkor kiszámoljuk az egyed fitnessértékét, majd a return utasítással visszaadjuk az egyedet.

A fitnessérték kiszámításához szükség van a kromoszómák dekódolására is, amelyet a decode() függvény végez. A metódus bemenetként egy egyedet vár, és egy útlistát ad vissza. Először létrehozunk egy üres utat, null járművel, majd elkezdjük olvasni az egyed kromoszómáját. Ha pozitív számot látunk, akkor egyszerűen hozzáadjuk a vevőt az úthoz. Ha viszont negatív számot olvasunk, akkor lezárjuk az utat, és beállítjuk az út járművét a negátnak megfelelő járműre, majd, ha még nincs vége a kromoszómának, létrehozunk egy új utat null járművel, és folytatjuk az olvasást.

A fitnessérték számítását a fitness() függvény végzi, amely az utak egy listáját várja, és egy double értéket ad vissza. A fitnessértéket az alábbi képlet alapján számoljuk:

$$\text{megoldás teljes költsége} + \text{kihagyott vevők száma} * \text{büntetés} \quad (4)$$

Vegyük példaként az alábbi listát: $\{\text{Route}(v_1, \{d, n_1, n_2, n_3, d\}), \text{Route}(v_2, \{d, n_4, n_5, d\})\}$, ahol alsó indexben a járművek, illetve a vevők indexe található. A megoldáshoz tartozó kromoszóma ekkor: $\{1, 2, 3, -1, 4, 5, -2\}$.

3.3.2 Kezdeti populáció kialakítása

Az algoritmus első lépése a kezdeti populáció kialakítása. Ehhez egy random megoldást generálunk, majd ezt belerakjuk a *population* listába egészen addig, amíg el nem érjük a kívánt méretet.

A véletlen megoldás generálásához a `random_sol()` függvényt használjuk. Először vegyük a flotta egy random permutációját, és menjünk végig a járműveken. Minden v járműhöz létrehozunk egy r Route változót, amelyhez hozzárendeljük a járművet. Ha még van olyan vevő, akit nem látogattunk meg, akkor generáljunk egy random számot 0 és a vevők száma között. Legyen az ennek a számnak megfelelő indexű csúcs n , és nézzük meg, hogy r összигényének és n igényének összege meghaladja-e v kapacitását. Ha nem, akkor folytassuk egy új random csúccsal, ha viszont meghaladja, akkor zárjuk le az utat, majd tegyük bele a megoldást tartalmazó listába.

Mielőtt visszaadnánk a megoldást, futtassuk le rá a `correct()` metódust. Ez a függvény megkapja a ki nem szolgált vevők listáját, illetve a megoldást, ami egy Route lista. Először iteráljunk végig a vevők listáján, és nézzük meg, hogy van-e olyan út, ahova be tudjuk őket illeszteni. Egy n csúcsot akkor tudunk beilleszteni az r útba, ha r összkeresletének és n keresletének összege nem haladja meg az r -hez tartozó jármű kapacitását. Ha ez a feltétel teljesül, akkor az n csúcsot beillesztjük az r út végére.

A keresztező operátorok használatánál fontos, hogy a kromoszómák hossza megegyezzen, ezért a kódolás megkönnyítése érdekében a kimaradt vevőket tegyük egy külön, *null* járművel rendelkező útba.

3.3.3 Kiválasztó operátorok

Ezután kiválasztjuk a szülőket egy kiválasztó operátorral. Ebből háromfélet implementáltam: egy rulett szelekciót, egy versenyalapú szelekciót, illetve egy első- n szelekciót.

A rulett szelekció az adatbázison nem ad jó eredményeket, egyrészt a megoldások relatív közelsége miatt, másrészt az algoritmus maximalizálásnál használatos, a HVRP-ben pedig minimalizálunk. Ebből kifolyólag ezt az operátort nem használtam, az implementációt ezért nem részletezem.

A versenyalapú szelekciónál először kiválasztunk n darab véletlen egyedet, az n értékét a függvény argumentumként kapja meg. Az n darab egyed közül a legkisebb fitnessértékűt belerakjuk a szülők listájába, majd folytatjuk a következő kiválasztásával, egészen addig, amíg a szülők listájának mérete megegyezik a populáció méretével.

Az első- n szelekciónál argumentumként kapunk egy i számot, amely megadja, hogy a populáció hanyadrészét szeretnénk szülőnek választani. A populáció méretét i -vel osztva megkapjuk az n számot. Ha a populáció mérete nem osztható i -vel, akkor n -t lefelé kerekítjük, hogy egy egész számot kapjunk. A populáció egyedeit a rátermettségük alapján csökkenő sorrendbe tesszük, és az első n egyedet i -szer átmásoljuk a szülők listájába. Könnyen belátható, hogy amennyiben a populáció mérete nem osztható i -vel, akkor a lefelé kerekítés miatt a szülők száma nem fog megegyezni a populáció méretével, ezért a kimaradó helyekre a legkisebb fitnessértékű egyedet másoljuk be, akár többször, ha ez szükséges. Végül a szülők listáját összekeverjük, hogy ne mindig ugyanazok a szülők szerepeljenek egymás mellett.

A versenyalapú szelekciónál be tudjuk állítani, hogy hány egyed közül válasszon az operátor, ezzel szabályozni tudjuk a konvergencia gyorsaságát, illetve tudjuk olyan egyedekre is alkalmazni, melyek fitnessértékei nagyságrendjükhöz képest közeliak. Az első- n szelekciónál is állíthatjuk, hogy a populáció mekkora részét szeretnénk használni, viszont az operátor használatával hamarabb elveszítjük a populáció diverzitását.

3.3.4 Keresztező operátorok

A kiválasztás után páronként keresztezzük a szülőket. Minden keresztező operátor bemenete kettő szülő egyed, kimenete kettő utód. Az egyszerűség kedvéért az implementáció leírásakor csak az első utód létrehozását részletezem, de a második utód is hasonlóan jön létre, csak a két szülő szerepe felcserélődik. Az utód kromoszómáját c -vel, az első szülőét p_1 -gyel, a második szülőét p_2 -vel jelöljük. A korábban tárgyalt három keresztező operátort implementáltam.

A részleges leképezésű keresztezés az adatbázis példáin túl nagy változásokat okoz, a jó megoldásokat a legtöbb esetben csak rontja, ezért nem használtam a tesztelésnél, az implementációt ezért nem részletezem.

A sorrend-alapú keresztezést (OX) az `ox_crossover()` függvény valósítja meg. Első lépésként kijelölünk két keresztezési pontot, i -t és j -t, valamint létrehozuk a c -t reprezentáló tömböt. Itt p_1 -et és p_2 -t nem tömbként, hanem listaként tároljuk. Ezután az i és j közötti géneket átmásoljuk p_1 -ből c -be. Azokat a géneket, amelyek már benne vannak c -ben, keressük meg a p_2 -ben és ezeket vegyük ki belőle. Ezzel kapunk egy listát, amelyben csak azok a gének szerepelnek, amelyek nincsenek benne c -ben, pontosan olyan sorrendben, mint p_2 -ben. A c szabad pozícióira illesszük be a p_2 -ben maradt géneket sorrendben.

Példaként vegyünk két 9 génből álló kromoszómát:

$$p_1 = \{1, 5, 8, 3, 4, 9, 2, 6, 7\} \text{ és } p_2 = \{3, 8, 5, 9, 7, 1, 4, 2, 6\},$$

és legyen a két keresztezési pont $i = 2, j = 5$. Először másoljuk át a pontok közötti géneket p_1 -ből, ekkor:

$$c = [0, 0, 8, 3, 4, 9, 0, 0, 0] \text{ és } p_2 = \{5, 7, 1, 2, 6\}$$

A maradék géneket illesszük be p_2 -ből:

$$c = [5, 7, 8, 3, 4, 9, 1, 2, 6]$$

A ciklus alapú keresztezést (CX) `cx_crossover` metódus valósítja meg. A p_1 és p_2 itt listák, c egy tömb. Első lépésként kiválasztunk egy keresztezési pontot, i -t, illetve létrehozuk a c -t reprezentáló tömböt. Ezután megkeressük a ciklust, amelyben az i pozíción szereplő érték szerepel. Ehhez létrehozunk egy listát, amelyben a ciklus elemeinek pozícióját tároljuk, és első elemként hozzáadjuk a keresztezési pontot. Szükségünk lesz egy változóra, amelyben az aktuális pozíciót tároljuk, legyen ez j . Legyen p_1 értéke i -ben $p_1[i]$, ekkor j új értéke $p_1[i]$ indexe p_2 -ben. A j új értékét ezután adjuk hozzá a ciklus elemeit tároló tömbhöz, és folytassuk addig ezt az iterációt, amíg i és j értéke megegyezik, ekkor a ciklus minden elemének indexe szerepel a listában. Ezután végigmegyünk c minden elemén, és ha a pozíció benne van a ciklusban, akkor a p_1 génjét másoljuk át, ha nincs benne, akkor a p_2 -ét.

A példában dolgozzunk a két korábbi kromoszómával, és legyen a keresztezési pont $i = 3$, majd keressük meg a ciklust:

$$p_1 = \{1, 5, 8, 3, 4, 9, 2, 6, 7\}$$

$$p_2 = \{3, 8, 5, 9, 7, 1, 4, 2, 6\}$$

A vonalak mutatják a leképezéseket a két szülő között, amelyek meghatározzák a ciklust. A ciklus elemeit másoljuk p_1 -ből, a többi elemet p_2 -ből:

$$c = [1, 8, 5, 3, 7, 9, 4, 2, 6]$$

Az OX és CX operátorok hasonló eredményeket adnak, de legtöbb esetben a sorrendalapú keresztezés valamennyivel jobb végső megoldást ad, ezért ezzel dolgoztam tovább.

3.3.5 Mutációs operátorok

A keresztezés után az utódokat egy p valószínűséggel mutáljuk. A mutációs operátorok bemente és kimenete is egy-egy egyed.

A csere mutációnál generálunk két véletlen számot, i -t és j -t. Egy segédváltozóba belerakjuk az egyed kromoszómájának értékét i helyen. Ezután az i -edik gént állítsuk át a j -edik gén értékére, majd a j -edik gén értékét a segédváltozóra. Ezzel megcseréltük a két gént.

A keverő mutációhoz szintén két véletlen számra lesz szükségünk, ezek i és j . Vegyük az egyed kromoszómájának i és j közötti szakaszát, majd a Collections.shuffle metódussal véletlenszerűen keverjük össze a szakasz elemeit.

Az inverziós mutációnál is szükségünk lesz i -re és j -re, ezek itt is véletlen egész számok. Hasonlóan a keverő mutációhoz, vesszük a kromoszóma i és j közötti szakaszát, és erre alkalmazzuk a Collections.reverse utasítást.

A tapasztalat azt mutatja, hogy a feladatomra a keverő és az inverziós mutáció is túlságosan rongálja a megoldást, ezért a csere mutációt használom.

3.3.6 Az utak helyreállítása és értékelése

A keresztezés és a mutáció eredményezhet érvénytelen megoldásokat. Bár mindkét esetben permutációs feladatokhoz való operátorokat használunk, tehát minden csúcs továbbra is pontosan egyszer fog szerepelni a kromoszómában, sérthetjük a HVRP korlátozásait. Ezért szükség van a megoldások helyreállítására, ahol ellenőrizzük, hogy minden út összkereslete a kiszolgáló jármű kapacitásánál nem nagyobb. Erre a célra az evaluate() függvény szolgál.

Először a decode() metódussal átalakítjuk a kromoszómát utak listájára, valamint létrehozunk egy N listát, amiben azokat a vevőket tudjuk tárolni, amelyek sértik a

korlátozásokat. Ezután végigmegyünk a megoldás útjain egyesével, és amennyiben az út összkereslete nagyobb, mint a jármű kapacitása, kivesszük az utolsó csúcsot az útból, egészen addig, amíg az állítás igaz, és ezeket a csúcsokat hozzáadjuk az N -hez.

Ezután futtatjuk a `correct()` metódust az N vevőhalmazra, és a módosított megoldásokra. Ez ugyanaz a függvény, mint amit a kezdeti megoldások generálásához használunk, a cél, hogy azokat a csúcsokat, amelyek nincsenek benne egyik útban sem, belerakjuk olyan utakba, ahova még beférnek. Ha a függvény lefutott, akkor a `code()` metódussal újra lekódoljuk a megoldást, és visszaadjuk az egyedet.

3.3.7 Az algoritmus paraméterei

A genetikus algoritmusnak számos olyan paramétere van, amely befolyásolja az algoritmus működését. Ezeket fogom bemutatni.

Az osztály *rand* változójában egy véletlen változót tárolunk, amelynek a konstruktorban meg tudunk adni egy seed-et. Minden véletlent igénylő utasításnál ezt a változót használjuk, tehát minden random szám, illetve minden permutáció generálása ezzel a változóval történik. Ennek azért van jelentősége, mert így garantálni tudjuk, hogy az eredmények reprodukálhatók legyenek.

Be tudjuk továbbá állítani a populáció méretét, illetve a generációk számát. A populáció méretének növelésével javítani tudjuk az algoritmus konvergenciáját, mivel, ha több utódunk van, több lehetőségünk van jó megoldások generálására, viszont ezzel növeljük a futási időt. A generációk száma adja az algoritmus megállási feltételét. Ha ez a szám túl kicsi az adott feladathoz, akkor nem kapunk jó megoldást, ha túl nagy, akkor felesleges számításokat végezhetünk, mivel az utolsó generációkban a legjobb megoldás fitnessértéke stagnál, vagy csak nagyon keveset változik, tehát nem kerülünk közelebb az optimumhoz.

Fontos paraméter a mutációs ráta is, ezzel állítjuk be a mutáció valószínűségét az utódokon. A mutáció nagyon fontos, hogy az algoritmus ne ragadjon lokális maximumba, ezért fontos a jó érték megtalálása. Ha ez a szám túl kicsi, akkor lassíthatja a konvergenciát, esetlegesen beragadást tapasztalhatunk, ha viszont túl nagy, akkor elronthatja a jó megoldásokat, ezzel szintén lassítva a konvergencia sebességét. A mutációs rátát növelhetjük az algoritmus futása közben, ezzel segítjük a diverzitás fenntartását.

3.4 Megoldások javításai

A megoldások minőségét javíthatjuk, ha a savings módszer eredményére további keresést végzünk. Ez lehet egy lokális keresés, de használhatjuk a genetikus algoritmust is.

3.4.1 Savings javítása lokális kereséssel

A lokális keresést a LocalSearch osztály valósítja meg. Az osztály tárolja a legjobb (*best*), illetve az előző iteráció legjobb megoldását (*original*) egy-egy Route listában.

Az algoritmus alapvetően iterációnként próbálja javítani a megoldást, lépésenként egy csúcs áthelyezésével, vagy két csúcs cseréjével. Ehhez egy `swap_all()` és egy `move_all()` függvényt használ, majd az ezek által megtalált legjobb megoldást elmenti az *original* változóba. Az algoritmus akkor áll meg, ha az adott iterációban nem tudott javítani a megoldáson, vagy eléri az előre megadott iterációs számot.

A `swap_all()` függvény végigmegy az összes (i, j) útpáron, valamint ezeken belül az összes (n_i, n_j) csúcspáron, ahol n_i egy i -beli csúcs, n_j pedig egy j -beli csúcs. Ha i és j ugyanaz az út, akkor a `swap()` függvényt használjuk, ha különbözőek, akkor pedig a `swap_between()` függvényt. Minden (n_i, n_j) párosra megnézzük, hogy a függvény által módosított útvonal jobb-e, mint a *best*, és ha igen, akkor ezt az értéket átállítjuk az aktuális útra.

A `swap()` függvénynek három paramétere van, az út: r , valamint a két cserélendő csúcs, n_1 és n_2 . Ebben az esetben nincs szükség semmilyen ellenőrzésre, mivel az út összkereslete a két csúcs cseréjével nem változik, tehát biztosan nem sértjük a HVRP korlátozásait. Ezért egy segédváltozó segítségével egyszerűen megcseréljük a két csúcsot.

A `swap_between()` metódus négy paraméterrel dolgozik. Mivel itt két különböző út között cseréljük a csúcsokat, szükségünk van az r_1 és r_2 utakra, valamint a hozzájuk tartozó n_1 és n_2 csúcsokra. A csere előtt ellenőrizni kell, hogy az utakhoz tartozó jármű kapacitását ne haladja meg a módosított út összigenye. Ha ez a feltétel nem sérül, akkor r_1 -ből eltávolítjuk n_1 -et, majd a helyére rakjuk n_2 -t, és hasonlóan járunk el r_2 -re.

A `move_all()` függvény a csúcsok áthelyezését végzi. Ehhez végigmegy az összes (i, j) útpáron, ahol $i \neq j$, majd az i út csúcsain is végigiterálunk. Az áthelyezést csak akkor végezzük el, ha nem sértjük a jármű kapacitás korlátozását. Az áthelyezés első lépéseként eltávolítjuk az n csúcsot r_1 -ből, és hozzáadjuk r_2 elejére. Ezután a `swap()` függvénnyel

megnézzük, hogy az úton belül hova kell elhelyeznünk az új csúcsot, hogy a legnagyobb javulást kapjuk. Ha megtaláltuk ezt a pozíciót, visszaadjuk ezt a módosított utat.

3.4.2 Savings javítása genetikus algoritmussal

A savings algoritmus eredményének javítására a genetikus algoritmust is használhatjuk. Ehhez azonban egy pár kisebb módosítást kell végeznünk a genetikus algoritmuson.

A populáció generálásánál eddig minden egyedet véletlenszerűen generáltunk, ezen módosítanunk kell. Bevezetünk egy véletlen értéket, legyen ez p , ez lesz annak az esélye, hogy egy egyed kromoszómájában a savings módszer eredménye szerepeljen. A p megfelelő kiválasztása fontos ebben a lépésben, mivel, ha túl sok ugyanolyan megoldásunk van, akkor nem lesz elég diverzitás a populációban, ha túl kevés egyed hordozza a megoldást, akkor pedig az első generációkban eltűnhet ez a megoldás.

A jobb megoldások elérése érdekében a mutációs rátán is módosíthatunk. Mivel a populáció egy része ugyanazokból a megoldásokból áll, ezért jóval hamarabb veszítjük el a diverzitást, mint általános esetben. Ennek megelőzésére megemeljük a mutációs rátát, ezzel próbáljuk ellensúlyozni a jelenséget.

A kiválasztásra az első- n operátort választjuk, mivel ezzel tudjuk garantálni, hogy nem veszítjük el a savings módszer eredményét, valamint az azokból létrejött jobb megoldásokat. A keresztezésre a sorrend-alapú keresztezés helyett a ciklus alapú keresztezést használjuk, mivel ez okozza a legkisebb mértékű változást.

4. Eredmények

4.1 Tesztkörnyezet

Az algoritmusok tesztelése a PyVRP könyvtárból letöltött adatokon történt. A [5] cikk, amelyből a példák származnak, egy egzakt branch-cut-and-price (BCP) algoritmust használ. A cikkben használt BCP algoritmust csak az 500 csúcs alatti példákon tesztelték, ezért csak ezekhez ismertek az eredményei.

A példák nevének felépítése a következő: $X + \text{csúcsok száma} + \text{alprobléma}$. A szakdolgozathoz csak HVRP feladatokat használtam, tehát a tesztadatokra csak az $X + \text{csúcsok száma}$ formátumban fogok hivatkozni

Az eredmények összehasonlításához fontos, hogy milyen mérőszámokat használunk. Az algoritmusok értékeléséhez az összköltséget, valamint az algoritmus futási idejét veszem figyelembe.

4.2 Savings algoritmus teljesítménye

	<i>BCP</i>	<i>Savings</i>	<i>Százalékos teljesítmény</i>
<i>X148</i>	80285	86574	+7.8%
<i>X223</i>	72251	85799	+18.7%
<i>X289</i>	141927	127795	+11%
<i>X317</i>	165763	179623	+8.4%
<i>X429</i>	91732	98638	+7.5%

4.1. ábra: A savings és a BCP algoritmus eredményei

A 4.1. ábra a BCP és a savings algoritmus eredményeinek összköltségét mutatja 5 különböző példán. A harmadik oszlopban az szerepel, hogy a savings módszer eredménye hány százalékkal tér el a BCP eredményétől.

A savings módszer eredménye átlagosan körülbelül 11%-kal tér el a cikkben tárgyalt branch-cut-and-price (BCP) algoritmusétól. Fontos megjegyezni, hogy a BCP egy egzakt algoritmus, a futási ideje nagyságrendekkel nagyobb, mint a savings módszeré, percek, akár órákat vehet igénybe.

A savings algoritmus teljesítményét nagyban befolyásolják a nagy igényű vevők, mivel ezek sokszor árván maradnak, nem férnek be egyetlen jármű útjába sem. Ennek oka, hogy az algoritmus implementációjában a jármű választása egy mohó döntés, mindig az első útba kerülő

csúcs alapján választjuk. Ezért, ha az algoritmus későbbi szakaszában találkozunk egy nagy keresletű csúccsal, akkor megeshet, hogy már nem áll rendelkezésre olyan jármű, melynek kapacitása nagyobb, mint ez a kereslet.

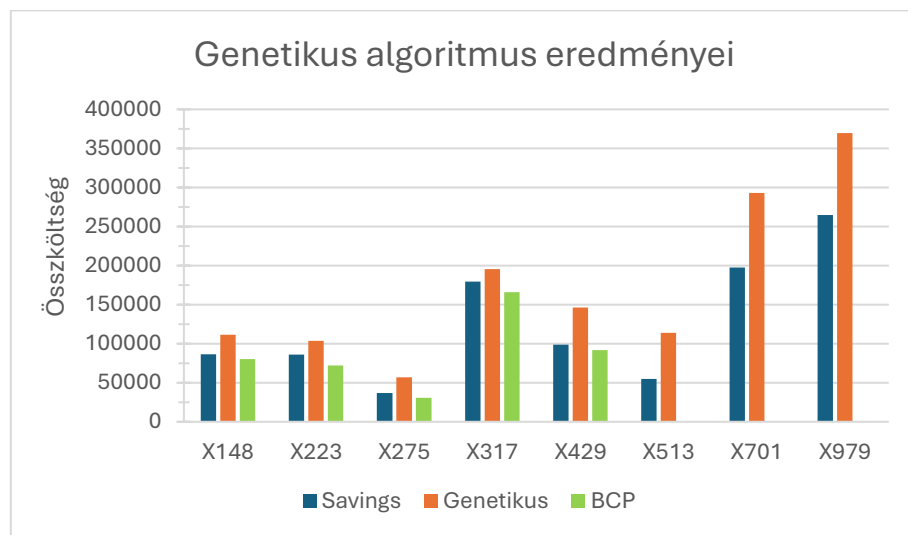
4.3 A genetikus algoritmus

A genetikus algoritmus tesztelésére 8 példát választottam, az egyik legkisebb méretű példától a legnagyobbig. Mivel az algoritmusban több helyen is használok véletlen változót, a kisebb példákon több seed-en is futtattam az algoritmust, de a megoldások között nagyon kis különbségek voltak, ezért a nagyobb feladatoknál egy seed-et használtam.

Minden feladaton 500 generációig futtattam az algoritmust, a populáció méretét a feladatban található vevők számának felére állítottam. A mutációs ráta kezdetben 0.025, vagyis 2.5% és minden generáció után a

$$\mu_{n+1} = \mu_n + \frac{0.001}{n} \quad (5)$$

képlet alapján növeljük, ahol n a populáció egyedeinek száma. A kiválasztásra a versenyalapú szelekciót használom, az operátor minden szülőt $n/20$ egyed közül választ. Ezután sorrend alapú keresztezéssel hozzuk létre az utódokat, és ezekre a csere mutációt alkalmazzuk.



4.2. ábra: A savings és a Genetikus algoritmus eredményei

A 4.2. ábra a genetikus algoritmus eredményeit mutatja nyolc különböző példán a savings módszer eredményeihez, valamint ahol elérhető, ott a cikkben látott eredményekhez viszonyítva. Az ábrán látszik, hogy bár a genetikus algoritmus a kisebb példákon megközelíti a savings algoritmus megoldásait, a nagyobb példákon egyre távolabb kerül. Ennek oka

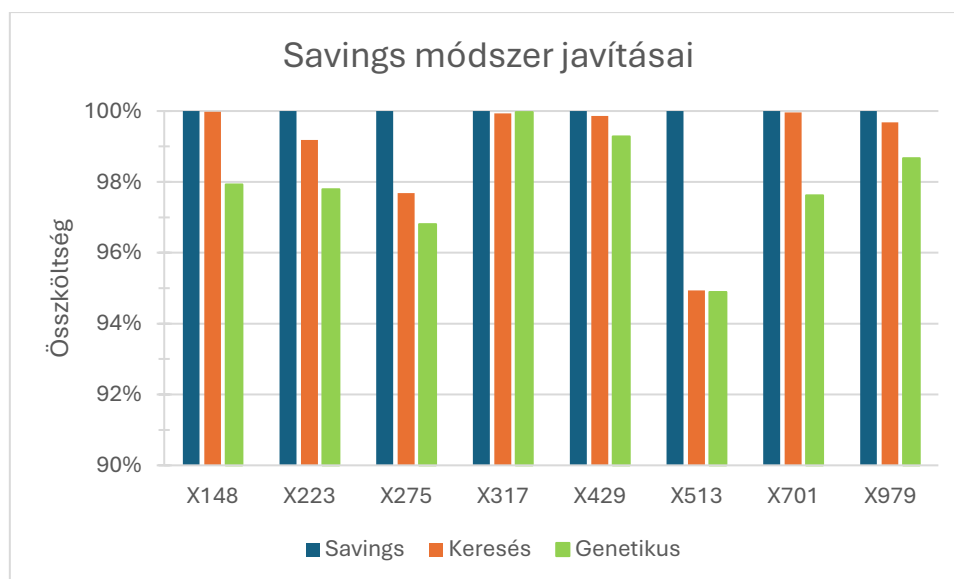
elsősorban az, hogy a genetikus algoritmus minden példán 500 generációig futott, ami a kisebb példákon elégnek bizonyult, a nagyobb feladatok azonban ennél nagyobb generációs számot igényelnek.

Annak, hogy a genetikus algoritmus nem éri el a savings módszer megoldásainak minőségét részben az lehet az oka, hogy a genetikus algoritmust teljesen véletlen megoldásokból indítjuk el.

4.4 Savings javításai

A lokális keresés alapvetően addig fut, amíg egy lépésben tud javítani a megoldáson. Az összehasonlíthatóság és a magas időigény miatt maximum 15 iterációt engedtem meg.

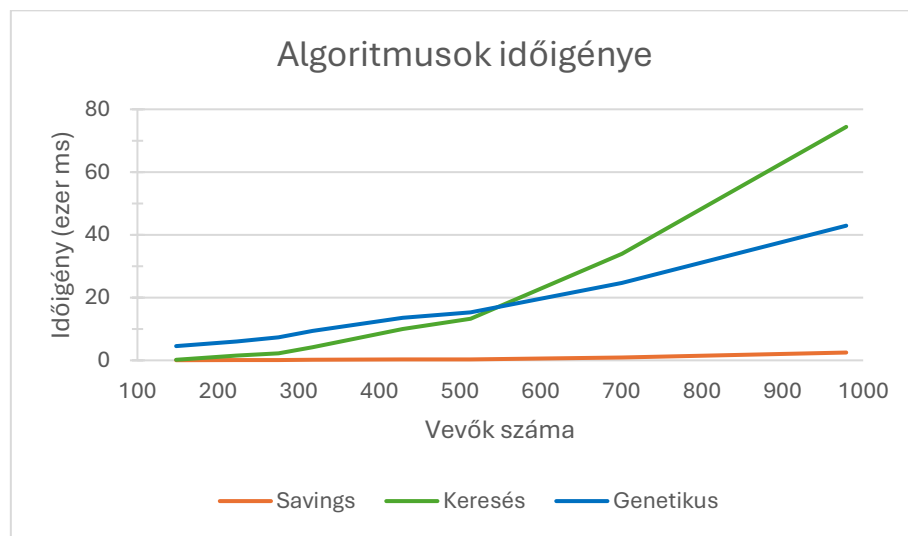
A genetikus algoritmusnál itt is 500 generációt engedtem meg, a populáció viszont fixen 100 egyed, mivel a növelés itt nem okozott jelentős javulást. A mutációs ráta kezdetben 0.15, és ugyanazon képlet alapján frissítjük, mint az előző esetben. Itt az első-n szelekciót használjuk, és a populáció első harmadát választjuk szülőnek. Bár az első-n szelekció jobban rontja a diverzitást, ebben az esetben szükségünk van rá, mivel csak ezzel a szelekciós operátorral tudjuk garantálni, hogy ne veszítsük el a kezdeti savings megoldásokat. A szülők kombinálására a ciklus alapú keresztezést alkalmazzuk, mivel ez kisebb változást okoz a kromoszómában, és mivel a savings módszer közel áll a legjobb ismert megoldáshoz, ezért ebben az esetben ez a legkedvezőbb választás. Mutációra a csere mutációt használjuk.



4.3. ábra: A savings javításainak eredményei

A 4.3. ábrán a savings módszer eredményei, valamint az ezeken futtatott lokális keresés, illetve genetikus algoritmus eredményei láthatók százalékos formában. Az eredményeket a savings algoritmushoz hasonlítjuk, ez a 100% a grafikonon.

A lokális keresés körülbelül 1.1%-ot tudott javítani a megoldásokon, míg a genetikus algoritmus átlagosan 2.2%-kal javította az eredményeket. Összességében elmondható, hogy azokon a példákon, ahol a savings megoldása közelebb került a legjobb ismert megoldáshoz, ott kevesebbet, ahol távolabb volt, ott többet tudtak javítani ezek a módszerek.



4.4. ábra: A javítások időigénye

Ezen az ábrán a savings algoritmus és a javítások futási idejét látjuk milliszekundumban mérve. A savings módszer időigénye a legnagyobb példán is 2500 ms alatti, messze ez a legkisebb. A lokális keresés görbéjén látszik, hogy bár alacsonyan indul, nagyobb a meredeksége, mint genetikus algoritmus görbéjének. A genetikus algoritmus tehát jobban skálázódik. Ennek oka, hogy a lokális keresés ebben az esetben gyakorlatilag egy brute-force megoldás, minden lehetséges módosítást kipróbál a megoldáson, míg a genetikus algoritmus a populációra és az operátorokra hagyatkozik.

4.5 Konklúzió

Az eredmények alapján megállapíthatjuk, hogy az adatbázis feladataira a savings módszer a legjobb választás. Alacsony futásidejű és az ismert legjobb értékhez közeli megoldást ad. A módszer eredményét bár kis mértékben tudjuk javítani, ez nem feltétlen éri meg, a javulás mértéke elhanyagolható a javító algoritmusok futási idejét figyelembe véve.

5. További lehetőségek

A savings algoritmus bár közel került a cikk [5] eredményeihez, sokszor hagyott árva csúcsokat. Egy okosabb járműválasztással azonban ezen még lehetne javítani.

A genetikus algoritmus eredményei sajnos elmaradtak a savings módszerétől, futási időben és a megoldások minőségében is. A futási idő javítása párhuzamos programozással lehetséges lenne, a szakdolgozatban viszont a megértésre és egy jól működő algoritmus implementálására törekedtem, így erre nem maradt idő.

A genetikus algoritmus megoldásait egy jobb feladatrepresentációval lehetne javítani. A szakirodalomban kapacitált VRP feladatokra sokszor a giant tour + split eljárást használják, ahol először figyelmen kívül hagyjuk a járműveket, és egyetlen hosszú útra keressük a legjobb megoldást, itt használhatjuk például a genetikus algoritmust, majd ezt az utat felbontjuk olyan kisebb utakra, amelyek teljesítik a járművek kapacitás feltételeit.

Az eredményeket genetikus algoritmus megállási feltétele is befolyásolja. Mivel a futtatásnál a generációk számánál nem vesszük figyelembe a feladat nagyságát, a kisebb gráfokon túl sok iterációt végezhetünk, míg a nagyobb, bonyolultabb feladatoknál nem eleget. Ennek megelőzésére beállíthatunk olyan megállási feltételeket, amelyek a generációk egyedeitől függenek. A feltételt köthetjük például a populáció diverzitásához vagy a generációk közötti legjobb megoldások különbségéhez. A tesztelésnél főleg a futási idő nagysága miatt nem alkalmaztam ezeket a feltételeket.

Irodalomjegyzék

- [1] Paolo Toth, Daniele Vigo (2002) The vehicle routing problem. *Society for Industrial and Applied Mathematics*
- [2] Said Elatar, Karim Abouelmehdi, Mohammed Essaid Riffi (2023), The vehicle routing problem in the last decade: variants, taxonomy and metaheuristics, *Procedia Computer Science 220 (2023)*, 398-404
- [3] G. Clarke, J. W. Wright (1964). Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. *Operations Research*, 12(4), 568-581
- [4] Barrie M. Baker, M.A. Ayechev (2003). A genetic algorithm for the vehicle routing problem. *Computers & Operations Research*, 30(5), 787-800
- [5] Artur Pessoa, Ruslan Sadykov, Eduardo Uchoa (2018). Enhanced Branch-Cut-and-Price algorithm for heterogeneous fleet vehicle routing problems. *European Journal of Operational Research*, 270(2), 530-543
- [6] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, Anand Subramanian (2017). New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257(3), 845-858
- [7] Shuguang Liu, Weilai Huang, Huiming Ma (2009). An effective genetic algorithm for the fleet size and mix vehicle routing problems. *Transportation Research Part E: Logistics and Transportation Review*, 45(3), 434-445
- [8] PyVRP könyvtár: <https://github.com/PyVRP>, Utolsó megtekintés: 2026. 04. 23.
- [9] Mayank Sethia – Clark-Wright Savings Algorithm: <https://www.kaggle.com/code/mayanksethia/clark-wright-savings-algorithm>, Utolsó megtekintés: 2026. 04. 23.
- [10] Genetic Algorithms: <https://www.geeksforgeeks.org/dsa/genetic-algorithms/>, Utolsó megtekintés: 2026. 04. 29.
- [11] Genetic Algorithms Tutorial: https://www.tutorialspoint.com/genetic_algorithms/index.htm, Utolsó megtekintés: 2026. 04. 29.
- [12] Wikipedia: <https://en.wikipedia.org>, Utolsó megtekintés: 2026. 04. 23.

Nyilatkozat

Alulírott Vizi Fanni programtervező informatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Számítástudomány Alapjai Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2026. 04. 29.

Vizi Fanni